Replicated Counter Service — Test Report

Course Distributed Systems — Assignment 2-5 & Midterm
Student _____
Repository _____ (submitted as dcc-assignment2.zip with full git history)
Date 9 October 2026

The system is a gRPC counter service built in three stages: one replica with retry-safe increments (Part A), Lamport-clock instrumentation (Part B), and three replicas where a write commits only after 2 of 3 acknowledgements (Part C). Every claim below cites an automated test from `tests/` or a captured log from `logs/`. Final suite result: **21 tests, 21 passed** (`logs/pytest_output.txt`).

1. Requirements under test

Task	Requirement	Verified by
A2 — dedup & retries	A reused idempotency key is not applied twice. Deadline 2 s; at most 3 retries on DEADLINE_EXCEEDED / UNAVAILABLE with the same key.	T02 duplicate_key_not_reapplied, T05 retry_after_timeout_is_safe, T10 retry_storm_same_key_applied_once, T11 request_without_key_is_rejected, F2 request_duplication, F3 timeout_then_retry_moves_once
A3 — concurrency	2 clients × 1,000 increments = exactly 2,000. One lock protects the dedup check and the mutation together.	T03 concurrent_increments_exact, T10 retry_storm_same_key_applied_once, `logs/no_lock_experiment.txt`
B1 — Lamport clocks	+1 before every local event; max(own, received)+1 on receive; timestamp carried in every message.	T13 lamport_clock_rules, T14 lamport_trace_respects_happened_before, `logs/b2/` (Section 7)
C1 — quorum writes	A write commits only with ≥ 2 of 3 acks. With fewer acks the client reports failure. Replicas converge.	T06 majority_commit_two_acks, T07 no_commit_below_majority, T08 replicas_converge, F1, F4–F7
C3 — failures	Crash, duplication and timeout+retry, each with an asserted invariant.	F1–F7 (Section 4)
C4 — performance	Median and p95 for 4 configurations, ≥ 2,000 requests each.	`tests/perf_benchmark.py`, `logs/perf_results.md` (Section 5)

2. Test environment

Item	Value
Machine	Apple M1, 8 logical CPUs, 8 GB RAM; client and replicas on the same host over localhost
OS	macOS 15.7.7 (build 24G720), arm64
Python	3.9.6 (system interpreter), virtual environment `.venv`
Packages	grpcio 1.80.0, grpcio-tools 1.80.0 (libprotoc 31.1), protobuf 6.33.6, pytest 8.4.2. Full `pip freeze` in Appendix A.
Commands	`python3 -m pytest -v`; `python3 tests/perf_benchmark.py`; `python3 tests/lamport_scenario.py` (see README.md)

3. Test cases

T-tests (`tests/test_counter.py`) run replicas in-process on ephemeral ports. F-tests (`tests/test_failures.py`) run every replica as a separate OS process. Each test follows arrange–act–assert. The "Actual" column is taken from the asserted values of the final run. The C2 file was also run 5 times in a row with 12/12 passing each time.

ID	Test / purpose	Expected	Actual	Result

T01	increment_applies_delta	incr(x, 5) → committed, value 5; get → 5	value 5, get 5	**PASS**
T02	duplicate_key_not_reapplied	2nd call with key k-1 → was_duplicate, value stays 5 (not 10)	5 / 5, duplicate = true	**PASS**
T03	concurrent_increments_exact	2 threads × 1,000 → exactly 2,000	2,000	**PASS**
T04	get_missing_counter	found = false, no exception	found = false, value 0	**PASS**
T05	retry_after_timeout_is_safe	1st reply delayed 1 s > 0.5 s deadline; retry with the same key; counter moves once	value 1, reply was_duplicate = true (answered by the retry)	**PASS**
T06	majority_commit_two_acks	1 of 3 replicas stopped → committed with 2 acks	committed, acks 2/3	**PASS**
T07	no_commit_below_majority	2 of 3 stopped → not committed, no value reported	committed = false, acks 1/3, new_value None; replica A holds 1 (anomaly, §6)	**PASS**
T08	replicas_converge	4 threads × 50 quorum writes → identical state on all replicas	{a:100, b:50, c:50} on A, B and C	**PASS**
T09	counter_ids_are_isolated (extra)	Writes to one counter do not change another	3 and 7	**PASS**
T10	retry_storm_same_key_applied_once (extra)	20 concurrent sends with one key → value 1 on every replica	1 on A, B, C	**PASS**
T11	request_without_key_is_rejected (extra)	Empty key → INVALID_ARGUMENT, nothing applied	INVALID_ARGUMENT, found = false	**PASS**
T12	negative_delta (extra)	+10 then −3 → 7	7	**PASS**
T13	lamport_clock_rules	local +1, send +1, receive max+1	1, 2, 11, 12	**PASS**
T14	lamport_trace_respects_happened_before	Per-process L strictly increasing; every RECV > received L	holds for 6 messages	**PASS**
F1	replica_crash_mid_request	See Section 4	committed 2/3, no exception	**PASS**
F2	request_duplication	See Section 4	5, 5, 5; 2nd was_duplicate	**PASS**
F3	timeout_then_retry_moves_once	See Section 4	1, 1, 1; acks 3/3 via retries	**PASS**
F4	crash_before_apply	See Section 4	committed 2/3; B exit ≠ 0	**PASS**
F5	crash_after_apply_before_reply	See Section 4	committed 2/3; B exit ≠ 0	**PASS**
F6	two_replicas_crash_no_commit	See Section 4	not committed, acks 1/3	**PASS**
F7	committed_writes_survive_single_crash	See Section 4	3 runs × 30 committed; survivors 30/30	**PASS**

Totals: 21 run, 21 passed, 0 failed, 0 skipped. Runtime 57.7 s, most of it in F-tests that start 3 processes each.

4. Failure-injection results

Route (a): the test kills a replica process with `proc.kill()`. Route (b): the replica's `--fault` flag simulates the failure. Logs for each test are in `logs/test_failures/<test>/`. F1–F3 are the mandated scenarios. F4–F7 form a small fault matrix: crash before apply, crash after apply, two crashes, and repeated random crashes.

ID	Scenario	Injected fault	Invariant asserted	Evidence
F1	Replica crash mid-request	B started with `--delay-ms 3000`; `proc.kill()` 0.5 s after the write starts	Write commits with 2 acks; no exception reaches the caller; A = C = 1	B log ends with `APPLY x -> 1` and no SEND: it died with the reply pending

F2	Request duplication	Same key `dup-key` sent twice to all replicas	2nd result was_duplicate; value 5 on A, B, C (not 10)	Each replica: `APPLY x -> 5`, then `DUP key=dup-key counter=x stays 5`
F3	Timeout + retry	All replicas `--fault slow-first --delay-ms 2500` (deadline 2 s)	Commit comes from retries (all acks duplicates); value 1 on every replica	A: `APPLY x -> 1` L=3, `DUP ... stays 1` L=13
F4	Crash before apply	B `--fault crash-before-apply`	Commit 2/3; B exited; A = C = 1	B log: RECV only, no APPLY
F5	Crash after apply, before reply	B `--fault crash-after-apply`	Commit 2/3; B exited; A = C = 1	B log: RECV, `APPLY x -> 1`, no SEND
F6	Two replicas down	B and C killed before the write	Not committed, new_value None; A did apply (= 1)	A: `APPLY x -> 1`, but client: acks 1/3
F7	No committed write lost	3 runs (seed 42): a random replica killed at a random write among 30	All 30 writes commit; both survivors hold exactly 30	assertion on survivor values

```
logs/test_failures/test_timeout_then_retry_moves_once/replica-A.log   (retry answered from the dedup store)
[replica-A] RECV   Increment(counter=x, delta=1) <- client                        L=2  (received L=1)
[replica-A] APPLY  counter=x -> 1                                                L=3
[replica-A] RECV   Increment(counter=x, delta=1) <- client                        L=12  (received L=11)
[replica-A] DUP    key=e5177bc9 counter=x stays 1                                L=13
[replica-A] SEND   IncrementReply(new_value=1) -> client                         L=14
logs/test_failures/test_crash_after_apply_before_reply/replica-B.log
[replica-B] RECV   Increment(counter=x, delta=1) <- client                        L=3  (received L=2)
[replica-B] APPLY  counter=x -> 1                                                L=4          (process exits here)
```

5. Performance (Task C4)

Method. `tests/perf_benchmark.py` starts fresh replica processes for each configuration. It sends 2,000 measured Increments, split evenly between 1 or 16 client threads, after a warm-up. The latency of each `client.incr()` call is recorded with `time.perf_counter()`. A quorum call returns as soon as 2 of 3 replicas acknowledge. p95 is the value at index ceil(0.95·n) of the sorted latencies (1-based, nearest rank). Each configuration was run 3 times. The table shows the mean of the 3 runs.

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.27	0.41	2000
Single replica, 16 clients	2.30	4.43	2000
Quorum (3 replicas), 1 client	0.45	0.64	2000
Quorum (3 replicas), 16 clients	5.68	14.60	2000

Configuration (mean ± std. dev., 3 runs)	Median (ms)	p95 (ms)	Throughput (req/s)
Single replica, 1 client	0.27 ± 0.01	0.41 ± 0.07	3485 ± 277
Single replica, 16 clients	2.30 ± 0.11	4.43 ± 0.48	6358 ± 377
Quorum (3 replicas), 1 client	0.45 ± 0.01	0.64 ± 0.16	2124 ± 124
Quorum (3 replicas), 16 clients	5.68 ± 0.55	14.60 ± 1.47	2285 ± 216

Interpretation. With one client, the quorum path costs about +0.18 ms at the median (1.7×). The client hands three RPCs to a thread pool and must wait for the second-fastest reply instead of the only one. It also pays for futures, thread hand-off and three Lamport clock updates under Python's GIL. With 16 clients the gap grows to 2.5× at the median and 3.3× at p95. Every write now creates three server-side requests (48 in flight instead of 16), and all of them compete for the same 8 cores of one laptop. Throughput confirms this: it falls from about 6,360 to about 2,290 req/s, close to the 3× extra work. On a real deployment with replicas on separate machines, CPU contention would mostly disappear. Latency would then be dominated by the network round trip to the second-closest replica. For a "like" counter, a p95 of about 15 ms is far below what a user notices. Surviving one replica crash without losing an acknowledged like is worth that cost. The quorum path stops paying off only when counts may be approximate. In that case a single replica with asynchronous replication would be cheaper.

6. Design notes

RPC semantics (A2). The design gives at-most-once execution of each mutation. Every logical operation carries an idempotency key, generated once with `uuid4()` before the first attempt. The replica stores the result of every applied key. Any later request with that key gets the stored result and `was_duplicate = true`, and the delta is not applied again. Bounded retries plus this store give exactly-once effect whenever the client eventually receives a reply. The key makes retries safe because it lets the server recognise a retry of work it has already done. Without the key, a retry would look like a new increment.

Synchronisation (A3). The server uses `grpc.server(ThreadPoolExecutor(max_workers=8))`. A single `threading.Lock` covers both the dedup lookup and the read-modify-write of the counter, so check-then-act is atomic. T03 passes (2,000). `logs/no_lock_experiment.txt` shows that removing the lock did *not* produce a lost update in 5 runs on CPython 3.9. The GIL makes the race window only a few bytecodes long. T03 therefore cannot prove correctness on its own; the lock is still required (see Limitations).

Partially applied failed write (C1). When only one replica acknowledges, the client reports failure, but that replica has already applied the write (T07, F6, `logs/demo_cli.log`: the write fails with acks 1/3, then reading replica-A returns 4). The failed write is therefore visible to readers of that replica, and it would become visible everywhere if the client later retried the same key. A consensus protocol such as Raft prevents this. The leader first appends the operation to a replicated log. A replica applies an entry to its state machine only after a majority has stored it (the entry is committed). Uncommitted entries can be overwritten by a later leader, so readers never see them.

Read consistency risk (C1). `Get` reads from a single replica. It can return a stale value from a replica that missed a write while it was slow or down. It can also return a value from a write that never committed. The B2 trace shows the first case: client-3 read `x = 0` while client-1's increment of x was already in flight.

7. Logical time and ordering analysis (Task B2)

`tests/lamport_scenario.py` starts replicas A, B and C. It then runs client-1 (incr x twice), client-2 (incr y twice) and client-3 (Get x, Get y) concurrently. Each process has its own clock and log file. The files are merged by L into `logs/b2/merged.log` (68 events). Figure 1 and Appendix B use the 28 events involving replica-A. Client sends to B and C are omitted, which is why client-1's L jumps from 1 to 10.

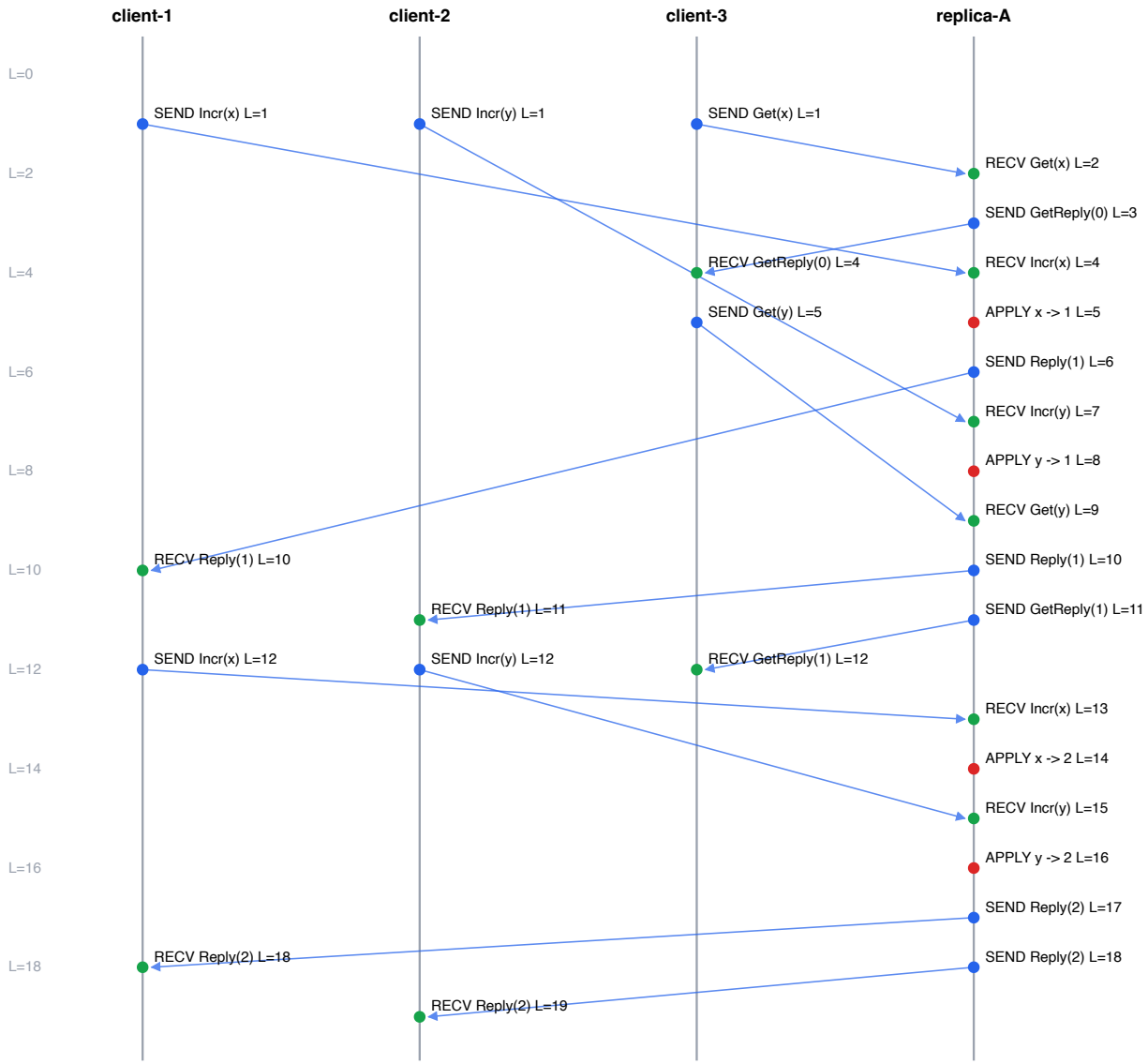


Figure 1. Happened-before diagram of replica-A and the three clients, drawn from `logs/b2/excerpt.log`. Time flows downward by L; arrows are messages; red dots are APPLY events.

Causally ordered pair 1: replica-A APPLY x→1 (L=5) → replica-A APPLY x→2 (L=14)

Chain: A `APPLY x->1` L=5 → (local) A `SEND IncrementReply -> client-1` L=6 → (message) client-1 `RECV` L=10 (received 6) → (local) client-1 `SEND Increment(x)` L=12 → (message) A `RECV` L=13 (received 12) → (local) A `APPLY x->2` L=14. The second increment was issued only after the first was acknowledged.

Causally ordered pair 2: client-2 SEND Increment(y) (L=1) → client-3 RECV GetReply(value=1) (L=12)

Chain: client-2 `SEND` L=1 → (message) A `RECV Increment(y)` L=7 → (local) A `APPLY y->1` L=8 → (local) A `RECV Get(y)` L=9 → (local) A `SEND GetReply(value=1)` L=11 → (message) client-3 `RECV` L=12. This chain explains why client-3 saw y = 1: client-2's write is in the causal past of that read.

Concurrent pair: client-3 RECV GetReply(value=0) (L=4) ∥ replica-A APPLY x→1 (L=5)

No path exists in either direction. After L=4, client-3's next message to A arrives at A only at L=9, after APPLY (L=5). After APPLY, A's next message to client-3 is the reply at L=11, which client-3 receives at L=12, after its event at L=4. The events are therefore concurrent even though 4 < 5. The values do tell us one thing: since L(a) < L(b), b cannot have happened before a. They do not tell us that a happened before b. Equal values carry no information either: client-1 and client-2 both log `SEND` at L=1 and L=12, yet they never exchange messages.

Limitation removed by vector clocks. With Lamport clocks, L(a) < L(b) does not imply a → b, so concurrent events cannot be told apart from causally ordered ones. Vector clocks satisfy V(a) < V(b) if and only if a → b, so concurrency is detected directly as incomparable vectors.

8. Known limitations

1. **State is in memory only.** A crashed replica that restarts comes back empty, with neither counters nor dedup store, and nothing brings it up to date (no log shipping). F7 tests crashes without restarts for this reason.
2. **Failed writes can be partially applied and are visible** (Section 6). There is no rollback, and a single-replica `Get` may return stale or uncommitted values.
3. **The dedup store grows without bound** (one entry per key forever). A real service needs a TTL or a per-client sequence window.
4. **T03 is necessary but not sufficient:** without the lock it still passed 5/5 runs on CPython (Section 6). A deterministic race test would need an injected pause between read and write.
5. **Benchmark realism:** one machine, loopback network, and a Python client limited by the GIL. Absolute numbers do not transfer to a networked deployment. In the B2 scenario the three clients are threads with separate clocks, not separate OS processes.

9. Reflection questions

1. Request lost vs reply lost. The client observes only that no reply arrived before the deadline. A lost request, a lost reply and a slow server all produce exactly that observation, and the client has no channel to ask which happened. The idempotency key makes the difference irrelevant: the retry carries the same key, so a replica that already executed the request returns the stored result instead of applying it again. F3 shows this: APPLY at L=3, then DUP at L=13, and the value ends at 1.

2. Smaller Lamport value ≠ earlier. Each process advances its own counter, and clocks are synchronised only when a message is received. A process that has handled few events has small values regardless of real time. In the trace, client-3's RECV (L=4) and A's APPLY x→1 (L=5) are connected by no message chain. The ordering $4 < 5$ is an artefact of how many events each process had seen; it is not evidence of order.

3. What three replicas tolerate. Any single replica crash is tolerated. This holds before applying (F4), after applying but before replying (F5), and mid-request (F1, F7): writes still commit and every committed write remains on at least one live replica. Safety ("no committed write is lost or applied twice") and liveness ("writes keep committing") both hold. With two replicas down, liveness is lost (F6: nothing can commit). Safety is also at risk: a committed write held only by the two crashed replicas is lost, because state is not persisted. The partial write on the survivor is visible without having committed.

4. Surviving restarts. The smallest change: before acknowledging, append each applied operation (key, counter, delta, result) to a file and fsync it; on startup, replay the file to rebuild counters and the dedup store. That is a write-ahead log. It anticipates Raft's persistent replicated log, which adds agreement on log order and catch-up of replicas that missed entries.

Appendix A — pip freeze

```
exceptiongroup==1.3.1
grpcio==1.80.0
grpcio-tools==1.80.0
iniconfig==2.1.0
packaging==26.3
pluggy==1.6.0
protobuf==6.33.6
Pygments==2.21.0
pytest==8.4.2
tomli==2.4.1
typing_extensions==4.16.0
```

Appendix B — Event-log excerpt used in Task B2 (`logs/b2/excerpt.log`)

```
[client-1] SEND   Increment(counter=x, delta=1) -> localhost:50051              L=1
[client-2] SEND   Increment(counter=y, delta=1) -> localhost:50051              L=1
[client-3] SEND   Get(counter=x) -> localhost:50051                             L=1
[replica-A] RECV   Get(counter=x) <- client-3                                    L=2  (received L=1)
[replica-A] SEND   GetReply(value=0) -> client-3                                 L=3
[client-3] RECV   GetReply(value=0) <- replica-A                                L=4  (received L=3)
[replica-A] RECV   Increment(counter=x, delta=1) <- client-1                     L=4  (received L=1)
[client-3] SEND   Get(counter=y) -> localhost:50051                             L=5
[replica-A] APPLY  counter=x -> 1                                                L=5
[replica-A] SEND   IncrementReply(new_value=1) -> client-1                       L=6
[replica-A] RECV   Increment(counter=y, delta=1) <- client-2                     L=7  (received L=1)
[replica-A] APPLY  counter=y -> 1                                                L=8
[replica-A] RECV   Get(counter=y) <- client-3                                    L=9  (received L=5)
[client-1] RECV   IncrementReply(new_value=1) <- replica-A                      L=10  (received L=6)
[replica-A] SEND   IncrementReply(new_value=1) -> client-2                       L=10
[client-2] RECV   IncrementReply(new_value=1) <- replica-A                      L=11  (received L=10)
[replica-A] SEND   GetReply(value=1) -> client-3                                 L=11
[client-1] SEND   Increment(counter=x, delta=1) -> localhost:50051              L=12
[client-2] SEND   Increment(counter=y, delta=1) -> localhost:50051              L=12
[client-3] RECV   GetReply(value=1) <- replica-A                                L=12  (received L=11)
[replica-A] RECV   Increment(counter=x, delta=1) <- client-1                     L=13  (received L=12)
[replica-A] APPLY  counter=x -> 2                                                L=14
[replica-A] RECV   Increment(counter=y, delta=1) <- client-2                     L=15  (received L=12)
[replica-A] APPLY  counter=y -> 2                                                L=16
[replica-A] SEND   IncrementReply(new_value=2) -> client-1                       L=17
[client-1] RECV   IncrementReply(new_value=2) <- replica-A                      L=18  (received L=17)
[replica-A] SEND   IncrementReply(new_value=2) -> client-2                       L=18
[client-2] RECV   IncrementReply(new_value=2) <- replica-A                      L=19  (received L=18)
```

Appendix C — Test run (`logs/pytest_output.txt`)

```
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Users/manapbeibarys/Desktop/распределенные системы/dcc-
tests/test_counter.py::test_increment_applies_delta PASSED               [  4%]
tests/test_counter.py::test_duplicate_key_not_reapplied PASSED           [  9%]
tests/test_counter.py::test_concurrent_increments_exact PASSED           [ 14%]
tests/test_counter.py::test_get_missing_counter PASSED                   [ 19%]
tests/test_counter.py::test_retry_after_timeout_is_safe PASSED           [ 23%]
tests/test_counter.py::test_majority_commit_two_acks PASSED              [ 28%]
tests/test_counter.py::test_no_commit_below_majority PASSED              [ 33%]
tests/test_counter.py::test_replicas_converge PASSED                     [ 38%]
tests/test_counter.py::test_counter_ids_are_isolated PASSED              [ 42%]
tests/test_counter.py::test_retry_storm_same_key_applied_once PASSED     [ 47%]
tests/test_counter.py::test_request_without_key_is_rejected PASSED       [ 52%]
tests/test_counter.py::test_negative_delta PASSED                        [ 57%]
tests/test_counter.py::test_lamport_clock_rules PASSED                   [ 61%]
tests/test_counter.py::test_lamport_trace_respects_happened_before PASSED [ 66%]
tests/test_failures.py::test_replica_crash_mid_request PASSED            [ 71%]
tests/test_failures.py::test_request_duplication PASSED                  [ 76%]
tests/test_failures.py::test_timeout_then_retry_moves_once PASSED        [ 80%]
tests/test_failures.py::test_crash_before_apply PASSED                   [ 85%]
tests/test_failures.py::test_crash_after_apply_before_reply PASSED       [ 90%]
tests/test_failures.py::test_two_replicas_crash_no_commit PASSED         [ 95%]
tests/test_failures.py::test_committed_writes_survive_single_crash PASSED [100%]
============================== 21 passed in 57.73s ==============================
```